SDOBase(SDK)使用说明

版本号：1.0

发布日期：2012-02

盛大在线
WWW.SDO.COM

目 录

1. 发布包 3

1.1 **包结构** 3

1.2 **包内容说明** 3

2. 配置文件说明 4

2.1 INIT.CONF 4

*2.1.1 **配置项说明*** *4*

*2.1.2 **配置文件*** *4*

2.2 LOG.PROPERTIES 5

2.3 **服务描述文件** 5

2.4 KEYWORDID.XML 6

3 接口文件说明 6

3.1 **应用接口** 6

*3.1.1 **创建应用接口函数**(sdoCreateCommonHandler)* *7*

*3.1.2 **初始化函数**(Initialize)* *8*

*3.1.3 **退出函数**(Uninitialize)* *9*

*3.1.4 **同步发送函数**(SyncSendMsg)* *10*

*3.1.5 **异步发送函数**(ASyncSendMsg)* *12*

*3.1.6 **注册回调函数**(RegisterCallback)* *14*

*3.1.7 **获取唯一 ID 函数**(GetGUID)* *15*

*3.1.8 **释放应用接口函数**(Release)* *17*

*3.1.9 **获取请求序列号**(GetRpcId)* *17*

3.2 **消息接口** 18

*3.2.1 **创建请求消息函数**(sdoCreateMsg)* *19*

*3.2.2 **设置协议字段值函数**(SetValue)* *20*

*3.2.3 **获取协议字段值函数**(GetValue)* *22*

*3.2.4 **释放消息接口函数**(Release)* *23*

*3.2.5 **获取消息名称函数**(GetMsgName)* *24*

1. 发布包

1.1 包结构

```
|----------include
|             |
|             |---SdoBase.h
|----------services
|             |
|----------libs
|             |
|             |---libSDOBase.so(SdoBase.dll)
|             |---libSDOBase.a(SdoBase.lib)
|
|----------init.conf
|
|----------log.properties
|
|----------KeyWordId.xml
```

1.2 包内容说明

文件名称	文件说明
init.conf	**配置文件**
KeyWordId.xml	**动态配置** KeyValue **文件，约定配置服务器下发的动态配置的内容**
Services **目录下** xml	**服务描述文件**,SDK **需要服务文件才能实现相应的功能**
log.properties	**日志配置文件**
SdoBase.dll	Windows **动态导出库**
SdoBase.lib	Windows **静态导出库**
SdoBase.h	**接口头文件**

2. 配置文件说明

2.1 init.conf

2.1.1 配置项说明

配置项	说明	值
ServerAddr	授权配置服务器地址	101.227.1.22:8094（联调）
LocalPort	本地连接端口	30028
StackMsgTimeOut	消息超时时间	3(s)
HostName	计费终端的唯一标识	当为 0 时 使用计费终端的 IP 和端口作为唯一标识
MaxAckMsgCacheNumber	必答消息最大缓存数量	20480
MaxAckMsgCacheSize	单个必答消息最大长度	4096
BusinessMsgTimeOut	业务层消息超时时间	30
MaxMsgQueueSize	消息队列最大长度	100240
AutoUnLock	是否自动发送解冻消息	1：发送

2.1.2 配置文件

```
#配置服务器地址

#联调环境

ServerAddr=101.227.1.22:8094

#正式环境

#ServerAddr=114.80.132.120:8094;112.65.240.16:8094;101.227.2.27:8094;140.207.211.87:8094
```

```
#SDK 绑定的本地端口
LocalPort=10086

#标识 GS 的别名，如果本地端口不固定，这使用该项标记服务器
HostName=0

#网络层消息超时时间(s)
StackMsgTimeOut=20

#必答消息最大缓存数量
MaxAckMsgCacheNumber=20480

#单个必答消息最大长度
MaxAckMsgCacheSize=4096

#业务层消息超时时间
BusinessMsgTimeOut=30

#消息队列最大长度
MaxMsgQueueSize=100240

#是否自动发送解冻消息
AutoUnLock=1
```

2.2 log.properties

组件的日志配置文件，调用方不需要做修改

2.3 服务描述文件

SDOBase 需要相应的服务描述文件（如用户验证、计费）等才能完成相应功能。所有服务描述文件都是 SDOBase 使用的，调用者不需要做任何修改。

2.4 KeyWordId.xml

这个文件对于计费来说不需要修改

```
<?xml version="1.0" encoding="UTF-8" ?>
<KeyWordIdList>
    <KeyWordId>ServerAddr:1</KeyWordId>
    <KeyWordId>SignKey:2</KeyWordId>
    <KeyWordId>SocPwd:3</KeyWordId>
    <KeyWordId>SpId:4</KeyWordId>
    <KeyWordId>ServerId:5</KeyWordId>
    <KeyWordId>AreaId:6</KeyWordId>
    <KeyWordId>GroupId:7</KeyWordId>
    <KeyWordId>HostId:8</KeyWordId>
    <KeyWordId>SiteCode:9</KeyWordId>
    <KeyWordId>Ip:10</KeyWordId>
    <KeyWordId>Port:11</KeyWordId>
</KeyWordIdList>
```

3 接口文件说明

3.1 应用接口

该接口是 SDOBase **的应用接口，是调用方和** SDO **后台服务交互的基础，接口** API **如下表**:

序号	接口名称	描述
1	sdoCreateCommonHandler	**创建应用接口**
2	Initialize	**初始化应用接口(初始化只能调用一次)**
3	Uninitialize	**反初始化应用接口**
4	RegisterCallback	**注册回调函数**
5	GetRpcId	**获取请求序列号**

序号	接口名称	描述
6	ASyncSendMsg	异步发送消息
7	SyncSendMsg	同步发送消息
8	GetGUID	获取唯一 ID(例如:订单号)
9	Release	释放资源
10	SDOBaseCallBack	回调函数由调用方实现

3.1.1 创建应用接口函数(sdoCreateCommonHandler)

3.1.1.1 函数功能

创建一个应用接口对象

3.1.1.2 函数原型

SNDAAPI(ISDOCommonHandler*) sdoCreateCommonHandler()

3.1.1.3 参数说明

无

3.1.1.4 返回值

返回值为 NULL 则为失败，反之为成功。

3.1.1.5 范例

```
//下面这个示例是关于如何创建一个应用接口
int main()
{
    //创建
```

```
ISDOCommonHandler *    g_pBillingHandler = sdoCreateCommonHandler();

//使用。。。。。

//程序结束的时候释放
pBillingHandler->Release();
return 0;
}
```

3.1.2 初始化函数(Initialize)

3.1.2.1 函数功能

初始化应用接口，想后台服务器注册，建立 TCP 长链接。

3.1.2.2 函数原型

```
SNDAMETHOD(int) Initialize(IN const char *pConfigFile) PURE;
```

3.1.2.3 参数说明

参数名	类型	类别	说明	备注
pConfigFile	char *	传入参数	配置文件	请使用绝对路径

3.1.2.4 返回值

0 表示成功，非 0 表示失败，具体错误码如下表：

错误代码	描述信息
-10242001	加载配置文件失败

错误代码	描述信息
-10242002	加载服务描述文件失败
-10242003	向授权服务器请求配置失败
-10242004	本地连接超时
-10242008	服务器连接失败

3.1.2.5 范例

```
//下面这个示例展示了初始化应用接口的流程
int main()
{
    ISDOCommonHandler    * pCommonHandler = sdoCreateCommonHandler();
    int ret = pCommonHandler->Initialize("init.conf");
    if(nRet != 0)
    {
        printf("init failed\n");
        return -1;
    }

    //程序结束的时候释放
    pCommonHandler-> Uninitialize();
    pCommonHandler->Release();
    return 0;
}
```

3.1.3 退出函数(Uninitialize)

3.1.3.1 函数功能

释放应用接口占用的资源，在程序退出时候调用

3.1.3.2 函数原型

SNDAMETHOD(void) Uninitialize() PURE;

3.1.3.3 参数说明

无

3.1.3.4 返回值

无

3.1.3.5 范例

请参考 Initialize **函数的范例**

3.1.4 同步发送函数(SyncSendMsg)

3.1.4.1 函数功能

向后台服务发起同步请求

3.1.4.2 函数原型

SNDAMETHOD(int) SyncSendMsg(IN const ISDOCommonMsg* pRequest, OUT ISDOCommonMsg* pResponse, IN unsigned int dwTimeout = 0) PURE;

3.1.4.3 参数说明

参数名	类型	类别	说明	备注
pRequest	ISDOCommonMsg *	**传入参数**	**请求消息**	
pResponse	ISDOCommonMsg	**传入参数**	**用于响应返回**	

dwTimeout	unsigned int	**传入参数**	**超时时间**	**单位秒**

3.1.4.4 返回值

0－成功，非 0 为失败，具体错误码如下表：

错误代码	描述信息
-10242005	**打包 SP 信息失败**
-10242006	**打包请求消息失败**
-10242007	**解包响应消息失败**
-10242400	**请求非法**
-10242401	**未注册**
-10242403	**请求拒绝**
-10242404	**请求的服务未找到**
-10242405	**请求的消息不支持**
-10242408	**请求发送超时**
-10242410	**请求发送失败**
-10242504	**请求处理超时**

3.1.4.5 范例

//下面这个示例展示了发送请求的流程

```
int main()
{
    ISDOCommonHandler    *g_pBillingHandler = sdoCreateCommonHandler();
    int nRet = g_pBillingHandler->Initialize("init.conf");
```

```
    if(nRet != 0)
    {
        printf("init failed\n");
        return -1;
    }
    ISDOCommonMsg *pMsg = sdoCreateMsg(E_Request, "Billing.Reserve");

    //调用 SetValue 设置协议字段值。。。。。。

    ISDOCommonMsg*pResponse=sdoCreateResponseMsg(E_Response, "Billing.Reserve");
    int nResult = g_pCommonHandler->SyncSendMsg(pMsg, pResponse, 3);
    if(nResult < 0)
    {
        printf("SyncSendRequest failed\n");
        pMsg->Release();
        pResponse->Release();
        return -1;
    }

    //调用 GetValue 获取响应值。。。。。。

    //释放消息
    pMsg->Release();
    pResponse->Release();

    //程序结束的时候释放
    pBillingHandler-> Uninitialize();
    pBillingHandler->Release();
    return 0;
}
```

3.1.5 异步发送函数(ASyncSendMsg)

3.1.5.1 函数功能

异步向后台服务发送消息，通过回调函数来处理请求响应。

3.1.5.2 函数原型

SNDAMETHOD(int) ASyncSendMsg(IN const ISDOCommonMsg* pRequest, IN unsigned int dwRpcId, IN unsigned int dwTimeout = 0) PURE;

3.1.5.3 参数说明

参数名	类型	类别	说明	备注
pRequest	ISDOCommonMsg *	传入参数	请求消息	
dwRpcId	Unsigned int	传入参数	用于请求序列号	
dwTimeout	unsigned int	传入参数	超时时间	单位秒

3.1.5.4 返回值

返回值参照同步请求。

3.1.5.5 范例

```
//下面示例了异步请求的流程
#include<map>
using std::map;
map<unsigned int, ISDOCommonMsg *>g_mapRequestByRpcId;
int SNDACALLBACK callbackfun(OUT EMsgDirection eMsgType, OUT unsigned int dwRpcId,
OUT int nResult,  OUT const ISDOCommonMsg* pMsg)
{
//异步请求的回调
}
int main()
{
    ISDOCommonHandler  *g_pBillingHandler = sdoCreateCommonHandler();
    int nRet = g_pBillingHandler->Initialize("init.conf");
    if(nRet != 0)
    {
```

```
        printf("init failed\n");
        return -1;
    }
    g_pBillingHandler->RegisterCallback(“Billing”, callbackfun)l
    ISDOCommonMsg *pMsg = sdoCreateMsg(E_Request, "Billing.Reserve");

    //调用 SetValue 设置协议字段值。。。。。。

    unsigned int nRpcId = g_ pBillingHandler->GetRpcId()
    int nResult = g_pBillingHandler->ASyncSendMsg(pMsg, nRpcId, 3);
    if(nResult != 0)
    {
        printf("ASyncSendRequest failed\n");
        pMsg->Release();
        return -1;
    }
    //保存请求，等相应回来时做相应处理
    g_mapMsgByRpcId[nRpcId] = pMsg;

    //程序结束的时候释放
    pBillingHandler-> Uninitialize();
    pBillingHandler->Release();
    return 0;
}
```

3.1.6 注册回调函数(RegisterCallback)

3.1.6.1 函数功能

用于调用方向 SDK 注册服务的回调函数。

3.1.6.2 函数原型

SNDAMETHOD(void)RegisterCallback(IN const char *pServiceName, IN const SDOBaseCallBack ***pCallBack***) PURE;

3.1.6.3 参数说明

参数名	类型	类别	说明	备注
pServiceName	const char*	传入参数	服务名	
pCallBack	SDOBaseCallBack	传入参数	回调函数	

3.1.6.4 返回值

无

3.1.6.5 范例

见 3.1.5.5

3.1.7 获取唯一 ID 函数(GetGUID)

3.1.5.1 函数功能

用来生成订单号等唯一 ID

3.1.5.2 函数原型

SNDAMETHOD(void) GetGUID(OUT char szGUID[33]) PURE;

3.1.5.3 参数说明

参数名	类型	类别	说明	备注
szGUID	char []	传入参数	用于返回唯一 ID	

3.1.5.4 返回值

无

3.1.5.5 范例

```
int main()
{
	g_pBillingHandler = sdoCreateCommonHandler();
	int nRet = g_pBillingHandler->Initialize("init.conf", RequestCb, ResponseCb);
	if(nRet != 0)
	{
		printf("init failed\n");
		return -1;
	}

	ISDOCommonMsg *pMsg = sdoCreateMsg(E_Request, "Billing.Reserve");


	//调用 SetValue 设置协议字段值。。。。。。

	char szId[33] = {0};

	//获取 proxyTicket

	g_pBillingHandler->GetGUID(szId);
	pMsg->SetValue("ccr.sequenceId", szId);

	ISDOCommonMsg *pResponse = sdoCreateMsg(E_Response, "Billing.Reserve");
	int nResult = g_pCommonHandler->SyncSendRequest(pMsg, pResponse, 3);
	if(nResult < 0)
	{
		printf("SyncSendRequest failed\n");
		pMsg->Release();
		pResponse->Release();
		return -1;
	}


	//调用 GetValue 获取响应值。。。。。。


	//释放消息
```

```
        pMsg->Release();
        pResponse->Release();

    }
```

3.1.8 释放应用接口函数(Release)

3.1.8.1 函数功能

释放应用接口对象.

3.1.8.2 函数原型

SNDAMETHOD(void) Release() PURE;

3.1.8.3 参数说明

无

3.1.8.4 返回值

无

3.1.8.5 范例

请参考 sdoCreateCommonHandler **的范例代码**

3.1.9 获取请求序列号(GetRpcId)

3.1.9.1 函数功能

获取异步请求对应的序列号

3.1.9.2 函数原型

SNDAMETHOD(unsigned int) GetRpcId()PURE;

3.1.9.3 参数说明

无

3.1.9.4 返回值

返回序列号

3.1.9.5 范例

请参考 ASyncSendMsg **的范例代码**

3.2 消息接口

该节描述 SDOBase **的消息处理接口，是** SDK **输入输出的基础，接口** API **如下表**:

序号	接口名称	描述
1	sdoCreateMsg	**创建请求消息**
2	GetValue	**从消息中获取相应字段的值**
3	SetValue	**设置消息中相应字段的值**
4	GetServiceName	**返回消息的名称**
5	Release	**释放消息**

3.2.1 创建请求消息函数(sdoCreateMsg)

3.2.1.1 函数功能

当需要发起请求的时候调用 sdoCreateMsg**，创建请求消息，请求结束以后调用** Release **释放消息**

3.2.1.2 函数原型

SNDAAPI(ISDOCommonMsg*) sdoCreateMsg(IN EMsgDirection eMsgType， IN const char * szMsgName);

3.2.1.3 参数说明

参数名	类型	类别	说明	备注
szMsgName	char *	传入参数	消息名称	例如 Billing.CheckBalance

eMsgType	EMsgDirection	传入参数	消息类型	typedef enum
				{
				E_REQUEST=0xA1, //请求
				E_RESPONSE=0xA2 //应答
				}EMsgDirection;

3.2.1.4 返回值

如果返回值为 NULL,**表示失败**;**否则为成功**

3.2.1.5 范例

//**下面这段代码示范了如何创建一个** reserve **请求消息**

```
int    DoReserveRequest()
{
    //创建请求消息
    ISDOCommonMsg *pMsg = sdoCreateMsg(E_REQUEST, "Billing.CheckBalance");
    //调用应用接口进行发送… …
    unsigned int pRpcId = pCommonHandler->GetRpcId();
    int ret = pCommonHandler->ASyncSendMsg(pMsg, pRpcId);

    //使用结束以后释放消息
    pMsg->Release();
    return 0;
}
```

3.2.2 设置协议字段值函数(SetValue)

3.2.2.1 函数功能

根据协议字段的 key 设置相应的 Value, key 值请见下面的调用示例。

3.2.2.2 函数原型

```
SNDAMETHOD(void)    SetValue(IN const char* pKey, IN void * pValue, IN int nLen) PURE;
SNDAMETHOD(void)    SetValue(IN const char* pKey, IN const char* pValue) PURE;
SNDAMETHOD(void)    SetValue(IN const char* pKey, IN int nValue) PURE;
```

3.2.2.3 参数说明

参数名	类型	类别	说明	备注
pKey	char *	传入参数	协议字段名称	例如 ccr.proxyTicket
pValue	char *	传入参数	协议字段值	
nValue	int	传入参数	协议字段值	
nLen	int	传入参数	协议字段值的长度	

3.2.2.4 返回值

无

3.2.2.5 范例

```
int   DoReserveRequest()
{
    //创建 reserve 请求消息
    ISDOCommonMsg *pMsg = sdoCreateMsg(E_REQUEST, "Billing.CheckBalance");
    //对协议字段设置值
    char szId[33] = {0};

    g_pBillingHandler->GetGUID(szId);
    pMsg->SetValue("ccr.sequenceId ", szId);

    g_pBillingHandler->GetGUID(szId);
    pMsg->SetValue("ccr.orderId", szId);
    string str0rderId = szId;

    g_pBillingHandler->GetGUID(szId);
    pMsg->SetValue("ccr.reserveOrderId", szId);

    pMsg->SetValue("ccr.userIdentifier.userId", "3456");
    pMsg->SetValue("ccr.userIdentifier.userType", 1);

    pMsg->SetValue("ccr.eventTimestamp", "12345678");

    pMsg->SetValue("ccr.billingOrder.createDate", "2010-8-6");
    pMsg->SetValue("ccr.billingOrder.billingOrderItem[0].usage.usage", 1);
    pMsg->SetValue("ccr.billingOrder.billingOrderItem[0].usage.type", 1);
    pMsg->SetValue("ccr.billingOrder.billingOrderItem[0].usage.propsId", "4");

    pMsg->SetValue("ccr.billingOrder.billingOrderItem[0].costs[0].cost", 100);
    pMsg->SetValue("ccr.billingOrder.billingOrderItem[0].costs[0].type", 1);

    pMsg->SetValue("ccr.businessCode", "7898");

    pMsg->SetValue("ccr.terminalIdentifier.ipAddress", "127.0.0.1");
```

```
        pMsg->SetValue("ccr.terminalIdentifier.type", 1);
        pMsg->SetValue("ccr.terminalIdentifier.platform", "2");

    //调用应用接口进行发送… …


    //使用结束以后释放消息

    pMsg->Release();
    return 0;
}
```

3.2.3 获取协议字段值函数(GetValue)

3.2.3.1 函数功能

根据协议字段的 key **获取相应的** Value, key **值请见下面的调用示例。**

3.2.3.2 函数原型

```
SNDAMETHOD(int) GetValue(IN const char* pKey, OUT void ** pValue, OUT int * pLen) PURE;
SNDAMETHOD(int) GetValue(IN const char* pKey, OUT char ** ppValue) PURE;
SNDAMETHOD(int) GetValue(IN const char* pKey, OUT int *pValue) PURE;
```

3.2.3.3 参数说明

参数名	类型	类别	说明	备注
pKey	char *	传入参数	协议字段名称	例如 cca.proxyTicket
pValue	char **	传入参数	用于返回字段值	不必关心内存 由组件负责,在消息释放 的时候自动释放
pValue	int *	传入参数	用于返回字段值	
pLen	int*	传入参数	用于返回字段值的长度	

3.2.3.4 返回值

0：成功 , -1：失败

3.2.3.5 范例

//下面示例了获取字段值

```
int GetResponseValue(ISDOCommonMsg* pResponse)
{
	char *pValue;
	int	nValue;

	if(pResponse->GetValue("cca.proxyTicket", &pValue) == 0)
	{
		printf("ProxyTicket:%s\n", pValue);
	}
	if(pResponse->GetValue("cca.orderId", &pValue) == 0)
	{
		printf("OrderId:%s\n", pValue);
	}
	if(pResponse->GetValue("cca.failReason", &pValue) == 0)
	{
		printf("FailReason:%s\n", pValue);
	}
	if(pResponse->GetValue("cca.userIdentifier.userId", &pValue) == 0)
	{
		printf("UserId:%s\n", pValue);
	}
}
```

3.2.4 释放消息接口函数(Release)

3.2.4.1 函数功能

释放消息对象占用的资源。

3.2.4.2 函数原型

```
SNDAMETHOD(void) Release() PURE;
```

3.2.4.3 参数说明

无

3.2.4.4 返回值

无

3.2.4.5 范例

```
//下面这段代码示范了如何释放一个请求消息
int    DoReserveRequest()
{
    //创建请求消息
    ISDOCommonMsg *pMsg = sdoCreateMsg(E_REQUEST, "Billing.CheckBalance");
    //调用应用接口进行发送… …
    unsigned int pRpcId = pCommonHandler->GetRpcId();
    int ret = pCommonHandler->ASyncSendMsg(pMsg, pRpcId);

    //使用结束以后释放消息
    pMsg->Release();
    return 0;
}
```

3.2.5 获取消息名称函数(GetMsgName)

3.2.5.1 函数功能

获取消息对象的消息名称。

3.2.5.2 函数原型

```
SNDAMETHOD(char *)    GetMsgName() PURE;
```

3.2.5.3 参数说明

无

3.2.5.4 返回值

返回值为 NULL 则为失败，反之为成功

3.2.5.5 范例

```
//下面这段代码示范了如何获取请求消息的名称
int    DoReserveRequest()
{
      //创建 reserve 请求消息
      ISDOCommonMsg *pMsg = sdoCreateRequestMsg("Billing.Reserve");

      //调用应用接口进行发送… …
      char *pServiceName = pMsg-> GetMsgName()
      //使用结束以后释放消息
      pMsg->Release();
      return 0;
}
```